Release Report: Torrent File Upload Fix

Revision: 1 **Last modified:** 2026-06-06T00:00:00Z

Date: 2026-04-03
Severity: Critical (complete loss of .torrent file upload functionality)
Status: Fixed, tested, deployed

Problem Description

Users were unable to download any torrent opened from a `.torrent` file through the qBittorrent WebUI. When uploading a `.torrent` file, setting a destination path, and submitting, the download never appeared in the downloads list and never started. Magnet links and search plugin downloads continued to work.

Root Cause

The `download_proxy.py` proxy server (sitting in front of qBittorrent on port 7186) attempted to decode **all** POST request bodies to `/api/v2/torrents/add` as UTF-8 text at line 101:

```
body_str = body.decode("utf-8")
```

When a `.torrent` file is uploaded via the WebUI, the browser sends it as `multipart/form-data` containing **raw binary data** (the bencoded torrent file). This binary content is not valid UTF-8, causing a `UnicodeDecodeError`:

```
'utf-8' codec can't decode byte 0x81 in position 530: invalid start byte
```

The exception was caught by the generic `except Exception` handler at line 131, which returned an HTTP 500 error silently. Users saw no error message in the WebUI - the upload simply disappeared.

Why magnet links worked

Magnet URLs are sent as `application/x-www-form-urlencoded` POST data (plain text), which IS valid UTF-8. The proxy only intercepted these for RuTracker URLs; all other magnets passed through correctly.

Why search plugin downloads worked

Search plugin downloads go through a different code path (`nova2dl.py`) that the proxy specifically handles for RuTracker URLs.

Architecture Context

```
User Browser
    |
    v
:7186  download_proxy.py (Python HTTP proxy)
    |       - Intercepts RuTracker URLs -> downloads via nova2dl.py
    |       - Passes through all other requests
    v
:7185 qBittorrent WebUI (native)
```

The proxy exists to handle RuTracker authentication for torrent downloads. It intercepts all API requests but should pass through non-RuTracker traffic unchanged.

Fix Applied

File: `plugins/download_proxy.py`

Changes

1. **Added multipart detection method** `_is_multipart_file_upload()` that checks the `Content-Type` header for `multipart/form-data`.
2. **Added early bypass for file uploads:** When a multipart file upload is detected, the proxy immediately passes the request through to qBittorrent without attempting to decode the body.
3. **Added binary body fallback:** Even if Content-Type check is bypassed, a `try/except UnicodeDecodeError` around the decode call ensures binary data doesnâ€™t crash the proxy.

Code diff (handle_request method)

Before:

```
def handle_request(self, body):
    try:
        path = urllib.parse.urlparse(self.path).path
        if path == "/api/v2/torrents/add" and self.command == "POST" and b
            body_str = body.decode("utf-8")  # CRASHES on binary torrent d
            params = urllib.parse.parse_qs(body_str)
            urls = params.get("urls", [""])[0]
            ...
```

After:

```
def _is_multipart_file_upload(self):
    content_type = self.headers.get("Content-Type", "")
    return "multipart/form-data" in content_type

def handle_request(self, body):
    try:
```

```
        path = urllib.parse.urlparse(self.path).path
        if path == "/api/v2/torrents/add" and self.command == "POST" and b
            if self._is_multipart_file_upload():
                logger.info("Multipart file upload detected, passing throu
                self.proxy_to_qbittorrent(body)
                return

            try:
                body_str = body.decode("utf-8")
            except (UnicodeDecodeError, ValueError):
                logger.info("Binary body detected, passing through directl
                self.proxy_to_qbittorrent(body)
                return

            params = urllib.parse.parse_qs(body_str)
            urls = params.get("urls", [""])[0]
            ...
```

Test Suite

File: `tests/test_torrent_file_upload.py`
Tests: 19 tests across 3 test classes

Test Torrent Files

Downloaded 4 Linux distribution torrent files with high seeder counts for testing:

File	Size	Source
`ubuntu-24.04-desktop-amd64.iso.torrent`	474 KB	releases.ubuntu.com
`ubuntu-20.04-desktop-amd64.iso.torrent`	325 KB	releases.ubuntu.com
`ubuntuserver.torrent`	247 KB	releases.ubuntu.com
`debian.iso.torrent`	50 KB	cdimage.debian.org

Located in: `tests/test_torrents/`

Test Classes

TestProxyInfrastructure (5 tests)

- qBittorrent direct accessibility (port 7185)
- Proxy accessibility (port 7186)
- Version match through proxy
- Transfer info passthrough
- Local torrent file availability

TestTorrentFileUpload (10 tests)

- Direct upload to qBittorrent (bypass proxy) - baseline
- Proxy upload returns â€œOk.â€ (not 500)
- Uploaded torrent appears in qBittorrent list
- Uploaded torrent has valid download state
- Magnet URL passthrough through proxy
- **All 4 distro torrent files upload successfully through proxy**
- Upload with custom save path
- Duplicate torrent detection through proxy
- **Regression: proxy never returns 500 for multipart uploads (5 iterations)**
- Preferences API passthrough through proxy

TestProxyRegression (4 tests)

- No UnicodeDecodeError on any torrent file (tests all 4)
- Binary torrent with all byte values (0-255) doesnâ€™t crash proxy
- Proxy logs contain no errors after uploads
- Largest torrent file (474 KB) uploads without issues

Test Results

```
Ran 19 tests in 47.951s - OK

ALL TORRENT FILE UPLOAD TESTS PASSED
The download proxy correctly handles .torrent file uploads.
Tested with 4 distro torrent file(s).
```

Tests verified on freshly restarted containers (full stop/start cycle).

Files Changed

File	Change
`plugins/download_proxy.py`	Fixed multipart file upload handling
`tests/test_torrent_file_upload.py`	New comprehensive test suite (19 tests)
`tests/test_torrents/`	New directory with 4 Linux distro torrent files

Deployment

Containers restarted and verified: - `qbittorrent` - qBittorrent v5.1.4-r2-ls444 on port 7185 - `qbittorrent-proxy` - Python proxy on port 7186

Manual Testing

The WebUI is now available at: **http://localhost:7186**
Credentials: `admin` / `admin`

To manually verify: 1. Open http://localhost:7186 in browser 2. Login with admin/admin 3. Click the â€œ+â€ icon or drag a `.torrent` file onto the window 4. Set a destination path (e.g., `/downloads/`) 5. Click â€œAddâ€ - the torrent should now appear and start downloading

---

Running Tests

```
# Run the torrent file upload test suite
python3 tests/test_torrent_file_upload.py

# Run all existing test suites
./run-all-tests.sh

# Quick validation
./test.sh
```